Multimodal autonomous agent application

Multimodal autonomous agent application

- 1. Concept introduction
  - 1.1 What is an "autonomous agent"?
  - 1.2 Brief description of the implementation principle
- 2. Code analysis
  - Key code
    - 1. The core agent workflow (`largemodel/utils/ai_agent.py`)
    - 2. The task planning and the interaction with the LLM (`largemodel/utils/ai_agent.py`)
    - 3. The parameter handling and the data flow implementation (`largemodel/utils/ai_agent.py`)
  - Code analysis
- 3. Hands-on operation
  - 3.1 Configure the large model
    - 3.1.1 Configure the LLM platform (`yahboom.yaml`)
    - 3.1.2 Configure the model interface (`large_model_interface.yaml`)
  - 3.2 Start it and test the function (text input mode)
  - 3.3 Voice interaction
- 4. Frequently asked questions and solutions
  - 4.1 The behaviour of the agent is abnormal
    - Problem 1: the agent falls into an infinite loop, or it keeps running the same tool.
  - 4.2 The tool call fails
    - Problem 2: the agent planned the action correctly, but the tool execution failed.

> **Hardware requirement: a Jetson with a 4GB configuration cannot run the cases of this tutorial; please use a Jetson with 8GB of memory or more.**

---

1. Concept introduction

1.1 What is an "autonomous agent"?

In the `largemodel` project, the **multimodal autonomous agent** is the most advanced intelligent form. It no longer simply responds once to a single instruction of the user; instead it is able to **think, plan and call a series of tools autonomously to accomplish a complex goal**.

The core of this function is the `agent_call` tool or the **tool chain manager (`ToolChainManager`)** behind it. The autonomous agent starts when the user makes a complex request that cannot be completed with a single tool call.

1.2 Brief description of the implementation principle

The implementation of the autonomous agent in `largemodel` follows the mainstream industry **ReAct (Reason + Act)** paradigm. Its core idea is to imitate the way human beings solve problems, cycling between "thinking" and "acting".

1. **Thinking (Reason)**: when the agent receives a complex goal, it first calls a powerful language model (LLM) to "think". It asks itself: "To complete this goal, what should my first step be? Which tool should I use?" The output of the LLM is not the final answer but a plan of action.

2. **Acting (Act)**: the agent performs the corresponding action according to the thinking result of the LLM — it calls `ToolsManager` to run the specified tool (such as `visual_positioning`).
3. **Observing (Observe)**: the agent gets the result of the previous action (the "observation"), for example `{"result": "Found the cup, located at [120, 300, 180, 360]"}`.
4. **Thinking again**: the agent submits this observation result, together with the original goal, to the LLM for a second round of "thinking". It asks itself: "I have found the position of the cup; what should I do next in order to know its colour?" The LLM may generate a new plan of action, for example `{"thought": "I need to analyse the image of the area where the cup is located to determine its colour", "action": "seewhat", "args": {"crop_area": [120, 300, 180, 360]}}`.

This **thinking -> acting -> observing** cycle continues until the original goal has been reached; only then does the agent generate and output the final answer.

2. Code analysis

Key code

1. The core agent workflow (`largemodel/utils/ai_agent.py`)

The `_execute_agent_workflow` function is the main execution loop of the agent; it defines the core flow of "plan -> execute".

```
# From largemodel/utils/ai_agent.py

class AIAgent:
    # ...

    def _execute_agent_workflow(self, task_description: str) -> Dict[str, Any]:
        """
        Executes the agent workflow: Plan -> Execute.
        """
        try:
            # Step one: the task planning
            self.node.get_logger().info("AI Agent starting task planning phase")
            plan_result = self._plan_task(task_description)

            # ... (return early if the planning failed)

            self.task_steps = plan_result["steps"]

            # Step two: execute all the steps in order
            execution_results = []
            tool_outputs = []

            for i, step in enumerate(self.task_steps):
                # 2.1. Before the execution, handle the data references in the parameters
                processed_parameters = self._process_step_parameters(step.get("parameters",
{}), tool_outputs)
                step["parameters"] = processed_parameters

                # 2.2. Execute the single step
```

```
                step_result = self._execute_step(step, tool_outputs)
                execution_results.append(step_result)

                # 2.3. If the step succeeds, save its output for the later steps to
reference
                if step_result.get("success") and step_result.get("tool_output"):
                    tool_outputs.append(step_result["tool_output"])
                else:
                    # If any step fails, abort the whole task
                    return { "success": False, "message": f"Task terminated because step
'{step['description']}' failed." }

            # ... summarise and return the final result
            summary = self._summarize_execution(task_description, execution_results)
            return { "success": True, "message": summary, "results": execution_results }

        # ... (the exception handling)
```

2. The task planning and the interaction with the LLM (`largemodel/utils/ai_agent.py`)

The core of the `_plan_task` function is to build a precise Prompt that uses the reasoning ability of the large model itself to generate a structured plan of execution.

```
# From largemodel/utils/ai_agent.py

class AIAgent:
    # ...
    def _plan_task(self, task_description: str) -> Dict[str, Any]:
        """
        Uses the large model for task planning and decomposition.
        """
        # Dynamically generate the list of the available tools and their descriptions
        tool_descriptions = []
        for name, adapter in self.tools_manager.tool_chain_manager.tools.items():
            # ... (get the description of the tool from adapter.input_schema)
            tool_descriptions.append(f"- {name}({params}): {description}")
        available_tools_str = "\\n".join(tool_descriptions)

        # Build a highly structured planning Prompt
        planning_prompt = f"""
As a professional task planning Agent, please decompose the user task into a series of
concrete and executable JSON steps.

**# Available tools:**
{available_tools_str}

**# Core rules:**
1.  **Data passing**: when a later step needs to use the output of an earlier step, it
**must** reference it with the `{{{{steps.N.outputs.KEY}}}}` format.
    - `N` is the ID of the step (counting from 1).
    - `KEY` is the name of the specific field in the output data of the earlier step.
```

```
2.  **JSON format**: a JSON object must be returned strictly.

**# User task:**
{task_description}
"""

        # Call the large model for the planning
        messages_to_use = [{"role": "user", "content": planning_prompt}]
        # Note that the general text inference interface is called here
        result = self.node.model_client.infer_with_text("", message=messages_to_use)

        # ... (parse the JSON response and return the list of steps)
```

3. The parameter handling and the data flow implementation (`largemodel/utils/ai_agent.py`)

The `_process_step_parameters` function is responsible for resolving the placeholders and implementing the data flow between the steps.

```
# From largemodel/utils/ai_agent.py

class AIAgent:
    # ...
    def _process_step_parameters(self, parameters: Dict[str, Any], previous_outputs:
List[Any]) -> Dict[str, Any]:
        """
        Parses the parameter dictionary, finds and replaces all the {{...}} references.
        """
        processed_params = parameters.copy()
        # The regular expression matches the placeholders in the {{steps.N.outputs.KEY}}
format
        pattern = re.compile(r"\\{\\{steps\\.(\\d+)\\.outputs\\.(.+?)\\}\\}")

        for key, value in processed_params.items():
            if isinstance(value, str) and pattern.search(value):
                # re.sub with a replacement function handles all the placeholders that were
found
                # The replacement function looks up the value in the previous_outputs list
                processed_params[key] = pattern.sub(replacer_function, value)

        return processed_params
```

Code analysis

The AI Agent is the "central brain" of the system; it turns the high level and sometimes vague tasks of the user into a series of precise, ordered tool calls. Its implementation does not depend on any specific model platform; it is built on a general and extensible architecture.

1. **Dynamic task planning**: the core capability of the agent is the `_plan_task` function. It does not rely on hard coded logic; instead it generates the task plan dynamically by interacting with the large model.
    - **Self awareness and Prompt construction**: at the start of the planning, the agent first checks all

the currently available tools and their descriptions. It then packs the tool information, the task of the user and the strict rules (such as the format of the data passing) into a highly structured `planning_prompt`.
    - **The model as the planner**: this Prompt is sent to a general text large model. The model reasons based on the provided context and returns an action plan in the JSON format that contains several steps. This design is extremely extensible: when tools are added or modified in the system, the planning capability of the agent updates automatically and the code of the agent itself does not have to be changed.
2. **The tool chain and the data flow**: the tasks of the real world often need several tools to work together; for example "take a photo and describe it" needs the output of the "take a photo" tool (the path of the picture) as the input of the "describe" tool. The AI Agent implements this elegantly with the `_process_step_parameters` function.
    - **The data reference placeholder**: during the planning phase, the large model embeds special placeholders such as `{{steps.1.outputs.data}}` into the parameter values where the data has to be passed on.
    - **Real time parameter replacement**: in the main loop of `_execute_agent_workflow`, `_process_step_parameters` is called before each step is executed. It scans all the parameters of the current step with a regular expression and, as soon as it finds a placeholder, it looks up the corresponding data in the list of the outputs of the previous steps and replaces it in real time. This mechanism is the key to automating the complex tasks.
3. **Supervised execution and fault tolerance**: `_execute_agent_workflow` forms the main execution loop of the agent. It strictly follows the planned order of the steps, executes each action in turn and makes sure that the data is passed correctly between the steps.
    - **Atomic steps**: each step is treated as an independent "atomic operation". If any single step fails, the whole task chain is aborted immediately and the error is reported. This ensures the stability and the predictability of the system and avoids continuing in an error state.

In summary, the general implementation of the AI Agent shows an advanced software architecture: it does not solve the problem directly; instead it builds a framework that lets an external, general purpose reasoning engine (the large model) solve the problem. With the two core mechanisms of "dynamic planning" and "data flow management", the agent is able to orchestrate a series of independent tools into a complex workflow that can complete advanced tasks.

3. Hands-on operation

3.1 Configure the large model

This section selects the backend through the platform option in `yahboom.yaml` and fills in the corresponding model parameters in `large_model_interface.yaml`. The online example uses Tongyi Qianwen; the offline example uses the local Ollama.

3.1.1 Configure the LLM platform (`yahboom.yaml`)

Open the main configuration file:

```
vim ~/yahboom_ws/src/largemodel/config/yahboom.yaml
```

Set `llm_platform` in the existing `model_service.ros__parameters` parameter block. You only have to choose one of the two backends:

```
model_service:
  ros__parameters:
    language: 'en'
    llm_platform: 'ollama'  # The offline large model; change it to 'tongyi' when you use
the online model
    regional_setting: "international"
```

- Offline Ollama: set `llm_platform: 'ollama'`.
- Online Tongyi Qianwen: set `llm_platform: 'tongyi'`.

If these parameters already exist in the file, only modify the corresponding values and keep the other configuration; do not add duplicate YAML keys with the same name.

3.1.2 Configure the model interface (`large_model_interface.yaml`)

Open the model interface configuration file:

```
vim ~/yahboom_ws/src/largemodel/config/large_model_interface.yaml
```

Depending on the backend selected in the previous step, only fill in the matching group of parameters.

Online Tongyi Qianwen: first apply for the API Key as described in the online large model basic tutorial and confirm that the account has available quota; then fill in the Key and a model that supports image input:

```
qianwen_api_key: "Replace it with your own API-Key"
qianwen_model: "qwen-vl-max-latest"
```

Replace the placeholder with your own API Key. The autonomous agent needs a model that understands the task and that generates an executable tool call plan following the prompt; if the task contains an image analysis, the vision tool that is called additionally requires the model to support image input. Never publish a real API Key.

Offline Ollama: make sure the Ollama service is running and that a model that is able to follow the instructions and to return a structured task plan has been downloaded; if the agent task has to analyse an image, that model also has to support image input. Then fill in the service address and the name of the local model:

```
ollama_host: "http://localhost:11434"
ollama_model: "llava"  # Configuration example; replace it with the model name from ollama
list that suits the agent task
```

The model name has to match the output of `ollama list`. `llava` is only a configuration example and is not guaranteed to reliably generate the structured plan that the agent needs; choose the model according to the actual task and confirm with the test below that both the planning and the tool call succeed.

After saving both configuration files, restart `largemodel` so that the settings take effect. The online mode requires the Jetson to be able to reach the Tongyi service; the offline mode requires the local Ollama service to be available.

> The speed of the task inference of the offline large model depends on the computing power of the Jetson mainboard and is usually slower than the online large model; the number of the called steps and of the tools also affects the time the task takes.

3.2 Start it and test the function (text input mode)

1. **Start the `largemodel` main program (text mode)**:
   Open a terminal and run the following command:

   ```
   ros2 launch largemodel largemodel_control.launch.py text_chat_mode:=true
   ```

2. **Send the text instruction**:
   Open another terminal and run the following command,

   ```
   ros2 run text_chat text_chat
   ```

   Then start typing the text: "Based on the current environment, save the generated environment description as a txt document".

3. **Watch the result**:

   In the first terminal that runs the main program you will see the log output showing that the system received the text instruction, called the `aiagent` tool and then gave the prompt to the LLM; the LLM analyses the detailed steps of the tool calls. For the question above, it calls the seewhat tool to get the frame and then gives the frame to the LLM to analyse; the text that is parsed out is saved in the folder ~/yahboom_ws/src/largemodel/resources_file/documents.

3.3 Voice interaction

The AI Agent can hand the task obtained from the speech recognition to the LLM for the planning and then call the tools according to the plan. The LLM backend is selected with `llm_platform`; the ASR/TTS use an independent configuration:

```
model_service:
  ros__parameters:
    use_oline_asr: False  # True: online ASR; False: offline ASR
    useolinetts: False    # True: online TTS; False: offline TTS
```

Complete the service configuration as described in the ASR/TTS tutorials. Set the online LLM to `tongyi` and the offline LLM to `ollama`. Start the voice program:

```
ros2 launch largemodel largemodel_control.launch.py
```

After waking the device up, say "Based on the current environment, save the generated environment description as a txt document". Check the recognised text of the ASR, the log of the `aiagent` planning and of the tool execution, and confirm that the result file was written to `~/yahboom_ws/src/largemodel/resources_file/documents` and that the speaker announces it. The complex tasks put high demands on the instruction following and on the tool calling ability of the model; if you do not see the tool execution log, troubleshoot with the text mode first.

4. Frequently asked questions and solutions

4.1 The behaviour of the agent is abnormal

Problem 1: the agent falls into an infinite loop, or it keeps running the same tool.

Solution:

1. **Strengthen the Prompt**: add stronger restrictions to the ReAct Prompt, for example: "Do not repeat the same operation" and "If two consecutive observations are the same, try a different tool or end the task".
2. **Increase the iteration limit**: like the `max_turns` in the code example, set a maximum number of cycles to prevent an infinite loop.
3. **Switch to a more powerful LLM**: the logical capability of the agent depends to a large extent on the "intelligence" of the backend LLM. A more powerful model understands the task and plans better.

4.2 The tool call fails

Problem 2: the agent planned the action correctly, but the tool execution failed.

Solution:

1. **Check the single tool**: this is usually not a problem of the agent logic but a problem of the called tool itself (such as `seewhat` or `visual_positioning`). Troubleshoot it by following the tutorial of the corresponding tool.
2. **The error handling**: in the loop of the `ToolChainManager` there has to be a complete `try...except` mechanism that catches the exceptions of the tool execution and feeds the error message back to the LLM as the "observation result", so that it knows that the previous step failed and can plan again.